

Interactive Visualization of LLL and BKZ Algorithms using Euclidean Vector Representations in Cartesian Coordinate

Muhammad Imam Chaidar Mujaddid Al-Ghiffari and Amril Syalim

Faculty of Computer Science, Universitas Indonesia

Abstract. As the quantum computing starts to threaten the current cryptographic systems such as RSA, cryptographers around the world start to develop the Post-Quantum Cryptography (PQC) algorithms, most of them are lattice-based (for example ML-KEM and ML-DSA). To strengthen these PQC algorithms, cryptographers and cryptanalysts alike also developing LLL and BKZ algorithms to test the strength of these PQC algorithms. One of the problems with LLL and BKZ is the fact that they're relatively difficult to learn. Some online tools to help users to understand LLL and BKZ algorithms have been developed, but many of them only show the initial and the final result of the input. In this paper, we will build Latviz, an interactive tool to explain the processes behind LLL and BKZ algorithms with explanations consist of visualization and textual information. Through visualization using vectors on Cartesian coordinate with some explanation panels for the processes of both LLL and BKZ algorithms, we expect cryptography learners and enthusiasts alike to understand LLL and BKZ easier with Latviz. To measure the quality of Latviz, we will make an online usability test survey. The survey revealed that 67.8% of participants reported an increase of their LLL understanding and 64.3% participants also reported an increase of their BKZ understanding, all after they tried Latviz.

Keywords: Cryptography, Cryptanalysis, Post-Quantum, Lattice, LLL, BKZ

1 Introduction

Cryptanalysis is the technique of analysing and breaking a ciphertext to obtain the plaintext [1]. The history and the development of cryptanalysis itself can't be separated from the history and development of cryptography. With the onset of the development of quantum computing and subsequently quantum cryptanalysis, many commonly used modern cryptographic algorithms, especially RSA, are threatened by quantum cryptanalysis [2]. Which is why cryptographers around the world already started to develop cryptographic algorithms that can resist quantum cryptanalysis - the post-quantum cryptography (PQC) [2]. Most of PQC algorithms that are seen as promising are lattice-based. For example, Kyber and Dilithium and their standardized versions - the ML-KEM and ML-DSA

algorithms. At the same time, cryptanalysts and cryptographers also use these lattice reduction cryptanalysis algorithms in order to evaluate the strength of these lattice-based PQC algorithms to resist their greatest foe - none other than the lattice reduction cryptanalysis algorithms [3].

The most common lattice reduction technique is the LLL algorithm, which was already developed long before the advent of quantum computing, and obviously predates the PQC algorithms as well. A polynomial-time algorithm for finding a short and nearly orthogonal basis for a lattice, this algorithm can be used to solve the shortest vector problem (SVP) and the closest vector problem (CVP), which are essential in lattice-based cryptography. We could say that LLL is the foundation of the lattice reduction cryptanalysis algorithms, and to master the subsequent algorithms, mastering the LLL before is a crucial step.

For now, the most promising lattice-based cryptanalysis technique is the BKZ algorithm, which uses several LLLs on a single BKZ run [4]. Compared to LLL, BKZ can find even shorter vectors in a lattice and theoretically speaking, BKZ is a far greater threat to lattice-based PQC systems compared to LLL, which are no longer a major threat for these lattice-based PQC systems. To help learners learn LLL and BKZ algorithms, some online solver tools have been developed, for example `fpLLL`, SageMathCell, and `lll-attack-runner`.

The problems with these LLL and BKZ solvers are the fact that they are either only show the final result, less user-friendly compared to simpler matrix tools, doesn't have visualization, or doesn't show the mathematical processes between the initial state and the final result. The first problem is true for `fpLLL` [5] and `lll-attack-runner` [6] as both tools show only the final result. The second problem is true for SageMathCell as we can't just insert a "plain" matrix to process it, instead we have to wrap the matrix within either a Python code, a Sage code, or any code in at least one of languages available in the tool [7]. To put it simply, it isn't as practical as insert matrices in some simpler online matrix tools, such as Calculator.net, matrixcalc.org, or Desmos. And the third problem is true for `fpLLL` and SageMathCell, as they don't even have built-in visualization. SageMathCell could show every step of the LLL process, however, we need to manually modify the code that we insert in the tool, so we can't use the built-in method of `.lll()`, and we have to use our own code. For the fourth problem, all previously mentioned LLL and BKZ algorithms solvers have this problem. None of these solvers show the mathematical processes between the initial state and the final result. For `lll-attack-runner`, it shows only a table which shows the reduced basis as well a panel for the gained SVPs. For SageMathCell, there is no built-in library to show these steps.

So in short, these LLL and BKZ tools aren't practical compared to several online math tools, and they don't automatically provide visualization from the beginning to the final result nor that they provide the step-by-step explanation. Which is actually an important issue, because psychologically wise, humans are generally easier to process a combination of visual and textual information [8]. However, we should not ignore that how the users interact with them is also an important factor [9]. So we definitely need step-by-step explanations just like we

need the visualizations, within a system that lets the user to interact with the system from the input insertion to the output visualization. For the visualization part, it's clear that visualization through vectors can significantly help users to understand the process. Why? Because the basis concept for LLL and BKZ is lattice reduction. Which is a linear algebra concept. And it is hard to teach the concepts of linear algebra without vectors (or, to be more precise, vector geometry) visualization [10].

Our idea is that by visualizing LLL and BKZ algorithms from the beginning of the process until the final result with vectors on 2D and 3D Cartesian coordinates, showing the calculations behind the algorithms, as well some simple theoretical explanations of these algorithms, we can help the users to understand these algorithms easier. The inputs are in the form of matrices. We will develop a tool named "Latviz", that wouldn't just process LLL and BKZ algorithms, but also show the visualization from the initial state until the final result, with simplified processes shown to help users understand these two algorithms easier. Our main idea for the visualization is through the usage of vectors within a Cartesian field, both in 2D and 3D projections, as well several panels that showing the current process (and its explanation), Lovasz condition status, and a button that can be pressed to go to the previous and next steps. We hope that through these visualization and textual elements, users would have it easier to understand the mechanism of both LLL and BKZ algorithms, which are increasingly becomes more relevant for the security of our data.

2 Related Works

2.1 Learning with Errors(LWE)

Learning with Errors (often shortened to LWE) is very essential for modern lattice-based PQC algorithms, such as NIST's ML-KEM and ML-DSA [11]. Basically, this is a method to hide the information (e.g plaintext) by injecting randomized noises (errors) into a system of linear equations which represents the public matrix, which would make the attackers struggling to get the actual pieces of the information from the plaintext. To solve an LWE problem means solving a set of linear equations with errors hidden in the equations. The errors are often tiny.

For example, there is a public matrix A , our secret message or plaintext s , and a random small number to mess the equation e . Normally, without the latter, a cipher is usually defined as $C = As \pmod{q}$. With LWE however, the equation becomes $C = As + e \pmod{q}$. Without the small errors, we can solve the system of linear equations with Gaussian Elimination, and thus the encryption algorithm is easier to cracked.

There are two types of LWE :

- Search LWE. This approach is to find s - the secret key from a noisy set of linear equations.

- Decision LWE. This approach is to distinguish an equation with LWE from a pure random noise. Usually, a real LWE set of equations would actually at first look random, but the errors gained from the solutions (gained from lattice reduction algorithms, not Gaussian elimination) of these equations do usually cluster around the actual key.

As the best practice, the size of the randomized error must small enough to make it decipherable for the intended receivers of the encrypted data. Small errors also able to force the attackers to switch into lattice reduction problem (because the errors resulted from attempts to solve these linear equations using Gaussian elimination), which essentially the same as searching shortest vectors (the SVP problem). Which, of course forcing the attackers to convert the system of linear equations into a lattice.

2.2 Lattice

A lattice is defined as a set which contains every single linear combinations of vectors from a basis of $\mathbb{R}^n$. Therefore, we can state that a lattice is theoretically a discrete subset of vector space with the dimension size of n .

Let's say, we have L , which is a lattice, then :

- k is the rank or the dimension size of L
- $\{v_1, v_2, \dots, v_k\}$ is the basis of L .

L must have vectors of $v_1, v_2, \dots, v_k$ that each of these vectors are linearly independent and such that $L = \mathbb{Z}$ -linear span of $\{v_1, v_2, \dots, v_k\}$ [12].

Lattice is one of the key mathematic concepts of the Post-Quantum Cryptography, in fact, of the four PQC algorithms that have been selected by NIST in order to be standardized, three are based on lattice : ML-KEM (based on CRYSTALS-Kyber), ML-DSA (based on CRYSTALS-Dilithium), and FN-DSA (based on Falcon).

2.3 Lattice-based Post-Quantum Cryptography

The lattice-based PQC is a group of cryptographic algorithms that like its name, are based on lattices, and their security is dependent on the hardness on two computational problems that are considered hard even for quantum cryptanalysis algorithms (such as Shor's Algorithm) to solve in the polynomial time : Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP). Example for lattice-based PQC algorithms are :

- CRYSTALS-Kyber. Standardized by NIST into ML-KEM (Module Lattice-based Key Encapsulation Mechanism). This algorithm, just like its pre-standardized version, is a set of algorithms used for two parties in order to establish a shared secret key in a channel considered to be unsafe.
- CRYSTALS-Dilithium. Standardized by NIST into ML-DSA (Module Lattice-based Digital Signature Algorithm). It is a set of algorithms that is used to generate as well verify digital signatures.

- Falcon (Fast Fourier Lattice-based Compact Signatures Over NTRU). Standardized by NIST into FN-DSA. It's also a DSA like ML-DSA.

Because most lattice-based PQC are using LWE to make their linear equation systems noisier, the problem essentially becomes equivalent (or at least related) to finding unusually short vectors, finding close lattice points, or reducing the lattice basis into a more orthogonal form. This is how the lattice basis reduction plays a vital role - although it doesn't guarantee the lattice-based PQC algorithm to be cracked.

2.4 Lattice Reduction

Lattice reduction is a process to find shortest vectors or closest vectors in lattices [12]. Within a lattice reduction process, the bases considered to be "bad," highly non-orthogonal basis (where the vectors are long and meet at sharp angles) are transformed it into a better version of them, which are reasonably orthogonal (each of their respective vectors are perpendicular to each other, vectors meet at angles closer to 90 degrees, and are relatively short). Why do we need "good" bases? Because good bases make hard lattice problems computationally easier to be solved.

Hadamard's Inequality is a method to tell a basis is a good or bad one.

$$\text{Det}(L) \leq \|v_1\| \cdot \|v_2\| \cdot \dots \cdot \|v_n\| \quad (1)$$

This inequality clearly states that the determinant of a lattice is always less than or equal to the product of the lengths of its basis vectors. For a bad basis, the product of the vector lengths is drastically larger than the true determinant (the determinant of the lattice where the basis belongs to). For a good basis, the product of the vector lengths are very close to the true determinant of the lattice. A "perfect" basis essentially has the same product of vector lengths with their true determinant. To put it simple, the closer the product of the vector lengths of a basis to the value of its true determinant, the better the basis is.

So, with lattice reduction, we can pick any single bad basis and turn it into a better version of it.

The most known modern lattice reduction method is Lenstra-Lenstra-Lovasz lattice reduction algorithm, often shortened to LLL. Virtually all modern mainstream lattice reduction algorithms are based on the LLL framework. For example, the Block Korkine Zolotarev (BKZ) lattice reduction algorithm uses LLL as an important kind of steps of it.

2.5 Shortest Vector Problem

The SVP (Shortest Vector Problem), like its namesake, is the problem of finding a shortest and non-zero vector in a lattice [12]. For a vector in a lattice to be regarded as the SVP, a vector must be the shortest one (or the shortest ones), out of all vectors in the lattice, and it must be a nonzero vector.

SVP is essential for PQC lattice reduction algorithms because it can help to solve the hidden noises injected in linear equations (the LWE problem) used in PQC algorithms.

2.6 LLL

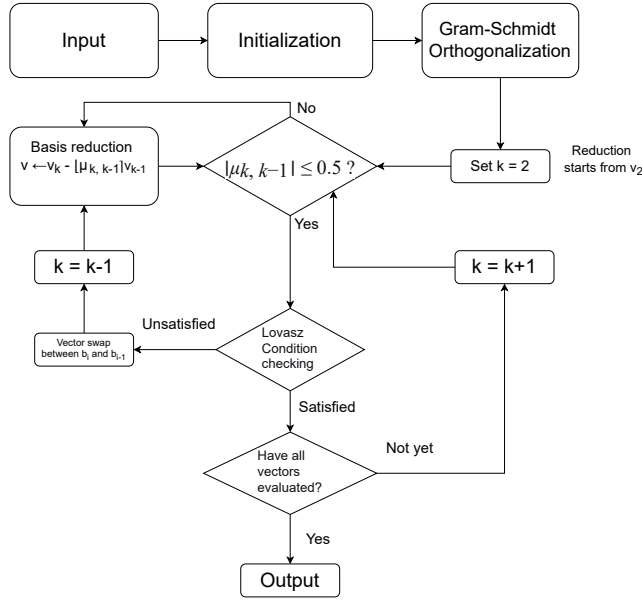


Fig. 1: The LLL algorithm

The LLL (Lenstra-Lenstra-Lovasz) cryptanalysis algorithm, developed by Arjen Lenstra, Hendrik Lenstra, and Laszlo Lovasz in 1982, is a lattice reduction algorithm. Although LLL in present days is largely ineffective to reduce lattices in many PQC algorithms such as ML-KEM and ML-DSA, the LLL is the foundation of modern lattice reduction cryptanalysis. For example, it serves as the foundation of BKZ, the upscaled version of LLL, by having several LLLs processes executed for a single BKZ process.

In the Fig.1, we can see how the algorithm works. The usual inputs are in the form of matrices, or a set linear equations. Either way, the acceptable form of input in our tool is matrix. However, in the general context, for a single LLL process, the input is in the form of a lattice basis L , which are in the form of a set of basis vectors $\{v_1, v_2, \dots, v_n\}$. Later for every vector within the set, they will be iteratively processed by Gram-Schmidt orthogonalization, then reduction or swap, and then examined whether the basis at the index fulfills

the Lovasz Condition or not. The iteration process, however, is controlled by a pointer variable k which starts from 2nd index to the n -th index (the last vector). In other words, the first vector only acts as a projection reference for Gram-Schmidt orthogonalization (it only acts as the initial orthogonal basis ($\mathbf{v}_1^* = \mathbf{v}_1$)) and size reduction processes of subsequent vectors, but the first vector is never individually subjected to the vector swap mechanism (unless during the first iteration, the second vector fails to meet the Lovász Condition checking) or Lovász Condition checking. After all subsequent vectors have gone through these processes, the loop ends and the algorithm stops.

Unless the initial lattice basis is already a highly orthogonal basis, we will obtain a more orthogonalized and shorter version of the basis. Thus, with a basis that is more orthogonal and shorter, we would have it easier for recovering the LWE small errors hidden in the linear equations that form the public matrix we want to crack.

As an important note, the δ value used in Lovász Condition ranges from 0.25 as the minimum value to 0.99 as the maximum value. A higher δ yields higher quality outputs at the expense of more iterations, and a lower δ leads to fewer iterations at the expense of lower quality outputs. In other words, a higher δ makes the recovery of the LWE errors easier.

2.6.1 Initialization

$$\mathbf{L} = \{v_1, v_2, \dots, v_n\}$$

We have L , a lattice basis, with n vectors within it. The L contains $\{v_1, v_2, \dots, v_n\}$. Assume L is a "bad" basis, then our goal is to make it more orthogonal and shorter.

2.6.2 Gram-Schmidt orthogonalization

$$\mathbf{v}_k^* = \mathbf{v}_k - \sum_{j=1}^{k-1} \frac{\langle \mathbf{v}_k, \mathbf{v}_j^* \rangle}{\|\mathbf{v}_j^*\|^2} \mathbf{v}_j^*$$

Right after the initialization, the algorithm processes the basis vectors sequentially. At the global scale (or the outer loop) in L , the algorithm performs Gram-Schmidt orthogonalization (GSO) for all vectors in L , produces the initial orthogonalized $\mathbf{v}_k^*$ and their corresponding projection coefficients $\mu_{k,j}$, with $1 \leq j < k \leq n$ before the main loop execution begins. Notice that for the first vector ($k = 1$), no projection is needed since it has no preceding vectors. Hence, $\mathbf{v}_1^* = \mathbf{v}_1$.

2.6.3 Main loop After the global Gram-Schmidt orthogonalization process, the index pointer k points at the second vector from the left (the vector of v_2), thus the main loop begins with the first iteration begins at $k = 2$.

2.6.4 Basis reduction loop The algorithm checks the size of the vector, and then performs a size reduction of the vector to ensure that the length of the projection coefficients satisfies :

$$|\mu_{k,j}| \leq 0.5$$

. To satisfy the condition, the algorithm uses an internal index pointer, j , whose sole purpose is to become the backward iterator in this basis reduction loop, and this loop starts from $j = k - 1$ down to 1. This ensures that the current target vector $\mathbf{v}_k$ is systematically reduced against all of its preceding vectors in the basis. The vector $\mathbf{v}_k$ is then iteratively updated using the following formula :

$$\mathbf{v}_k \leftarrow \mathbf{v}_k - \lfloor \mu_{k,j} \rfloor \mathbf{v}_j$$

Concurrently, to maintain mathematical consistency with the modified vector components, the algorithm must update $\mathbf{v}_k^*$ and the corresponding projection coefficients $\mu_{k,j}$, by fully recomputing the entire Gram-Schmidt orthogonalization sequence to get the updated $\mathbf{v}_k^*$ and $\mu_{k,j}$ values whenever $\mathbf{v}_k$ is modified inside this basis reduction loop.

The basis reduction loop continues until the backward iterator j has fully decreased past 1, signifying that the $\mathbf{v}_k$ has been checked and reduced against every preceding vector in the basis L . Once the basis reduction loop ends, the updated vector is then subjected to the core step of the LLL algorithm - the Lovász Condition checking.

2.6.5 The Lovasz Condition

$$\|\mathbf{v}_k^*\|^2 \geq (\delta - \mu_{k,k-1}^2) \|\mathbf{v}_{k-1}^*\|^2$$

Using the updated $\mathbf{v}_k^*$ obtained from the preceding basis reduction loop, the algorithm evaluates whether the current vector ordering satisfies the Lovász Condition or not.

Should the basis at the current iteration satisfies the Lovasz Condition, it will proceed to check whether all vectors have been examined. If that's the case ($k \geq n$), then the main loop terminates, signifying that the entire basis has been successfully reduced, and followed immediately by the termination of the algorithm altogether.

If not, then the iteration continues and k will point to the next vector ($k \leftarrow k + 1$), continuing the main loop until all vectors in the basis L have been fully processed.

However, if the basis fails to satisfy the Lovasz Condition at the current iteration, a vector swap mechanism is executed and followed by a re-evaluation of the basis sequence.

2.6.6 Vector swap If the current basis configuration does not satisfy the Lovász Condition at index k , it implies that the vector $\mathbf{v}_k$ has a significantly

shorter orthogonal projection compared to its predecessor, $\mathbf{v}_{k-1}$, which makes the ordering of the vectors within L becomes deficient. To fix this ordering deficiency, a vector swap mechanism must be executed. The order of the two adjacent vectors within the basis is interchanged as follows:

$$\mathbf{v}_{k-1} \leftrightarrow \mathbf{v}_k \quad (2)$$

Since this swap alters the geometric orientation and structural sequence of L , the previously calculated Gram-Schmidt vectors and coefficients become invalid from index $k - 1$ onwards. Consequently, the algorithm retreats its main loop iteration counter using the following step :

$$k \leftarrow \max(k - 1, 2) \quad (3)$$

This repositioning ensures that the counter shifts backward by one step, forcing the algorithm to recompute the local Gram-Schmidt orthogonalization and re-apply the basis reduction at this lower index before the Lovász Condition can be re-evaluated. As the important note, the max function serves as a logical guardrail to prevent k from accessing indices lower than 2.

2.6.7 Output After all of the vectors have been examined and L meets the Lovasz Condition requirement, the algorithm terminates and returns a more orthogonalized and shortened version of L . This way, with a better lattice basis, we would be able to recover hidden LWE errors embedded within the linear equations which form the public matrix we want to crack.

2.7 BKZ

The BKZ (Block Korkine-Zolotarev) lattice reduction algorithm was first proposed by Clauss-Peter Schnorr in 1987, and in 1994 he and Martin Euchner published the journal of the BKZ algorithm [13]. This lattice reduction algorithm, while based on LLL, is much more stronger compared to LLL. Nowadays the usage of BKZ in cryptanalysis is common, as the algorithm becoming increasingly important due to the emergence of the lattice-based post-quantum cryptographic algorithms [14].

2.7.1 Initialization and the first LLL

$$\mathbf{L} = \{v_1, v_2, \dots, v_n\}$$

We have L , a lattice basis, with n vectors within it. The L contains $\{v_1, v_2, \dots, v_n\}$. Like in LLL algorithm, our goal is to make L becomes more orthogonal and shorter. The first step here is to run a LLL process to the entire basis vectors within L .

$$\text{LLL}(\mathbf{L}, \delta)$$

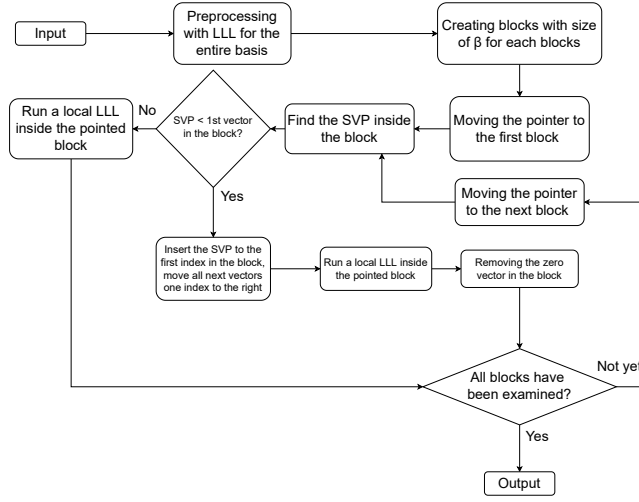


Fig. 2: The BKZ algorithm

The δ -value used in BKZ for all LLL executions within a BKZ algorithm execution is the same, and normally the δ value used is 0.99. This is done to ensure a more orthogonalized and shorter outputs for each LLL executions and the final output of the BKZ execution as well.

2.7.2 The main iteration After the first global LLL process has been completed, the algorithm begins the main iteration. While the LLL process iteratively process the vectors one-by-one across the entire basis (for a single iteration in LLL, only one vector is reduced), BKZ processes vectors within localized regions of the basis called blocks. Think of it as a pointer that points β neighbouring vectors for each iteration, and performs SVP searching, vector swapping, and localized LLL, all exclusively only for all vectors that are currently pointed. After the localized LLL is done, the pointer shifts just one vector to the right.

So in another and simpler words the so-called "block" structure in BKZ behaves more like a sliding window with a fixed block size β that traverses the lattice basis from left to right, and performs SVP enumeration, vector swapping, and LLL only for all vectors within the block.

2.7.3 Block formation For each single iteration, the BKZ algorithm selects a local block from the basis :

$$\mathbf{L}_{[i,h]} = \{v_i, v_{i+1}, \dots, v_h\}$$

in which

$$h = \max(i + \beta - 1, n)$$

Here, i is the starting index and h last index of the current block, while β is the fixed block size of the entire BKZ algorithm.

For example, we have $\mathbf{L} = \{v_1, v_2, v_3, v_4, v_5\}$ with $\beta = 3$. The first block would be $\mathbf{L}_{[1,3]} = \{v_1, v_2, v_3\}$ and after the first iteration ends, the next iteration's block would be $\mathbf{L}_{[2,4]} = \{v_2, v_3, v_4\}$. As we see, the "sliding window" only moves by one vector to the left, which is why the BKZ's blocks are overlapping to each other.

As an important note, since

$$\beta$$

-value is the symbolization of the block size, it means that the β -value determines how many neighboring vectors are included inside the current block during each iteration, and as such, the selection of the β -value has a big role on determining the quality of the BKZ's output. A smaller β produces smaller blocks, which means it produces a faster execution of the algorithm with the cost of the BKZ's output, while a bigger β resulted in slower execution but with better output.

2.7.4 Localized SVP enumeration After the block formation, the BKZ algorithm searches the shortest non-zero vector (SVP) inside the current block. Shall the SVP candidate v is deemed to be shorter enough compared to the currently examined basis vector (as mathematically stated below) :

$$\|v\| < \beta \|b_i\|$$

, then the SVP candidate v is deemed to be an improvement to the basis structure. Thus, the candidate v is inserted into the basis at the current block position $(v, b_i, b_{i+1}, \dots, b_h)$. Also usually known as nontrivial insertion. Should the condition $\|v\| \geq \beta \|b_i\|$ happens, then no insertion would be done, since the algorithm doesn't consider this case as an impactful improvement of the basis quality.

2.7.5 Localized LLL As the title of this section suggests, within the current block, a LLL process is executed, which only involves all vectors within the current block.

2.7.6 Output Once all blocks within the lattice basis have been examined and processed, and no further nontrivial insertion can be executed, the BKZ algorithm terminates. Like the LLL algorithm, it would produce a shorter and more orthogonalized lattice basis compared to the initial basis. And because the BKZ algorithm is formed from a global LLL, followed by the execution of localized LLL reductions and localized SVP enumerations in each of its blocks, the quality of a BKZ output is generally better (the output is expected to be much shorter and much more orthogonalized) than the output produced from a single LLL process.

Thus, with BKZ, recovering LWE errors hidden in the system of linear equations within a lattice-based post-quantum cryptographic system is much easier

to be done with BKZ. The only downside from a BKZ execution compared with LLL, is the computational cost, since a single BKZ process involves several LLL processes.

2.8 lll-attack-runner

lll-attack-runner is a Javascript-based interactive web application for running advanced lattice basis reduction algorithms such as LLL and BKZ. It is developed by Zachary Robert Bennett with the latest update was in January 2026. Because we aim to create a Javascript-based interactive web application for visualizing lattice-based cryptographic and cryptanalysis algorithms, in which two of them are LLL and BKZ, we will use lll-attack-runner as the backend for our visualization of the LLL and BKZ algorithms. However, since we want to visualize these two algorithms in an intuitive way which is easier to understand and more practical for cryptography learners, we will develop our own frontend for the visualization.

2.9 Herramienta de software para la visualización de la criptografía basada en retículas

Authored by Jaziel D. Flores Rodríguez and his fellow colleagues, this research paper was aimed to create a visualization tool for post-quantum cryptographic algorithms. The main focus was to graphically visualize mathematical concepts behind these post-quantum cryptographic algorithms, particularly lattice concepts which are the basis of these algorithms. The tool which was developed in this research, successfully visualizes these lattice-based post-quantum cryptographic algorithms within dimensions of 2, 3, 4, and 5. However, the LLL process for dimensions higher than 3 is still not implemented in this tool, and the BKZ algorithm is also not implemented in this tool. How we will visualize vectors in LLL and BKZ algorithms is inspired from this paper, which are in the form of arrows anchored to the origin point of $(0,0)$ in 2D visualization mode, and $(0,0,0)$ in 3D visualization mode, in an Euclidean Space [17].

3 Architecture

3.1 Overview

Within our source code repository, there is a segregation between the backend and frontend. The role of these backend source codes is to process the inputs inserted from the user through the interface at the frontend. And obviously the role of the frontend section, aside from getting the inputs from the user, is to do the entire visualization jobs. Because the visualization is our main priority, we can say that the frontend section is much more complex compared to the backend one.

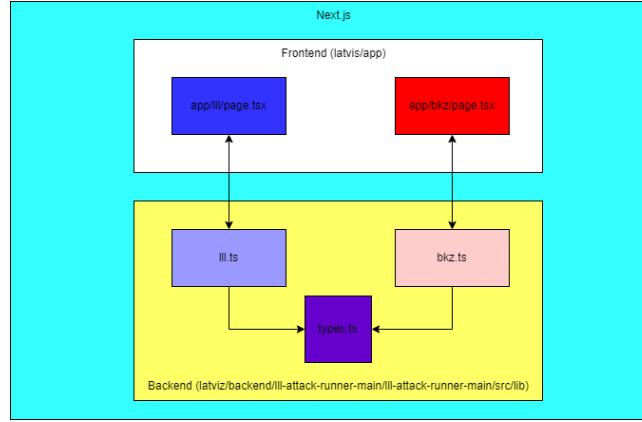


Fig. 3: Program structure of Latviz's LLL and BKZ visualizers

3.2 Backend

We're using the modified version of Zachary Robert Bennett's `lll-attack-runner` as the backend for our visualization of the LLL and BKZ algorithms. We only use three source codes from `lll-attack-runner`, which are `lll.ts`, `bkz.ts`, and `types.ts`. The `lll.ts`, per its filename, does the LLL algorithm process, and the same goes for `bkz.ts`. Both source codes need `LLLStep` interface from `types.ts`, and for `bkz.ts`, it also needs `LLRun` from `lll.ts`.

3.3 Frontend

Within the frontend section, in each of BKZ and LLL algorithm modules, each algorithm has its own folder. Which, in each of these folders, there is a single source code written in Typescript, with naming convention of the algorithm name written in lowercase. Each of these source codes are responsible for running the algorithm process which corresponds to their name. Within each of these source codes, we will use imported functions from Zachary Robert Bennett's `lll-attack-runner` source codes to run the algorithm process.

We will use Next.js, a React-based framework for building web applications, with the goal to create a user-friendly interface as much as we can for our visualization of the LLL and BKZ algorithms. The user will write their own matrix (or copy-and-paste it from any sources), insert needed values (for example, delta value for both LLL and BKZ algorithm and the block size for BKZ algorithm), and the frontend will pass the values inserted by the user into the backend for the core processing. As what we already explained before in the overview section, the frontend's responsibility is mainly for visualization part. And, for the visualization part, we will use D3 for the visualization part, combined with React to create matrices and vectors visualizations.

4 Visualization

4.1 Euclidean Vector Representations for the Visualization of LLL and BKZ

Vector visualization is a very essential as well as an intuitive concept to gain proper understanding of linear algebra concepts [15]. This also includes LLL and BKZ algorithms. And so, for both 2D and 3D visualization panels in each module of LLL and BKZ, we will depict the vectors extracted from the input (the matrix we inserted) as Euclidean vectors within 2D and 3D Cartesian coordinate systems.

The reason why we choose Euclidean vectors to represent the processes as well as the inputs of the LLL and BKZ is because both algorithms are obviously lattice reduction algorithms, which operate on the vectors within a Euclidean space. As such, visualizing both algorithms through Euclidean vector representations become a natural choice to show the behavior of lattice bases during the lattice reduction processes. Subprocesses within LLL and BKZ such as the Gram-Schmidt Orthogonalization, basis reduction, and vector swapping (LLL), as well as the localized LLLs (BKZ) are involving changes of length, direction, and angles of these vectors. To numerically map the Euclidean Space, we will use the Cartesian coordinate system, on the basis that lattice basis vectors are naturally represented numerically as coordinates in Euclidean space. Since the LLL and BKZ algorithms operate through geometric transformations of vectors, we could state that the Cartesian coordinate system is an intuitive way to visually display the changes of the properties of these vectors.

Within the Cartesian coordinate system we will use, whether it is the 2D or the 3D visualization, these vectors are depicted as arrows that are anchored to the origin point of $(0,0)$ in 2D Cartesian coordinate and $(0,0,0)$ in 3D Cartesian coordinate. The reason why we make the $(0,0)$ and the $(0,0,0)$ as the common origin points of all vectors within our visualization panel is to simplify the observation of geometric properties between basis vectors. Thus, by anchoring all vectors to these common origin points, users can directly compare the orientations and magnitudes of vectors without additional coordinate transformations or positional offsets. Conventionally, how the vectors are visualized in linear algebra and vector geometry in the linear algebra (which also includes lattice reduction algorithms like LLL and BKZ) are as arrows originating from the coordinate origin, which in our case, is $(0,0)$ for 2D visualization and $(0,0,0)$ for 3D visualization.

Each arrow has a different colouring scheme and is unique in terms of value, as every vector is unique (as they're linearly independent). No vectors would have more than one arrow that represents them (as they all must be linearly independent).

Thus, by representing basis vectors as Euclidean vectors inside Cartesian coordinate systems which all start from $(0,0)$ as the "anchor point", and with unique colour for each single vector, we expect the users to be able to visually observe how highly non-orthogonal bases gradually transform into shorter and

more orthogonal bases throughout the LLL and BKZ algorithms in an easier way.

4.2 Representation of Algorithmic Processes in the Visualization System

As what we already stated before, we will build our program based on the LLL and BKZ algorithms. Because our tool is a visualizer, it is obvious that it's necessary to transform the elements from the LLL and BKZ algorithms (such as initial matrix, vector reduction, vector swap, Lovasz status checking, final output, and so) into visual and textual elements. We think that some elements are better visualized, while some others are better shown in textual format.

4.2.1 Lattice Basis Representation of the Input Matrix

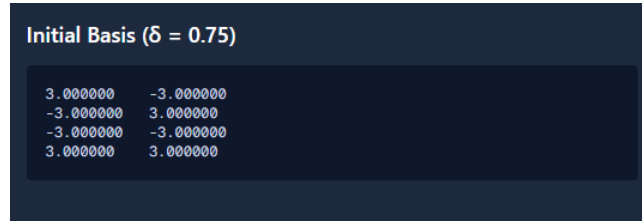


Fig. 4: The initial basis panel (LLL)

The whole visualization process begins after the user insert their matrix in the input field, and then clicked "Insert Matrix" button. Within the frontend system, the inserted matrix is parsed into a 2D array, which represents the basis of the inserted matrix. The outer array represents the entire matrix, while the inner array represents the individual basis vectors which are represented by spatial Cartesian coordinates. These basis vectors will be shown as vectors arranged vertically from top to bottom (just like their matrix counterparts) with the first vector is the topmost vector basis. For the LLL visualization module, the 2D array, alongside with the inserted δ -value will be passed into the runLLL function (which executes a LLL process) and runBKZ function (which executes a BKZ process) in lll.ts and in bkz.ts. For the BKZ visualization module, the 2D array, the δ -value, and the β -value will be passed into the runBKZ function in bkz.ts, which executes a BKZ process.

4.2.2 Euclidean Vectors Representation of the Basis Vectors As what we already stated before, the basis vectors extracted from the input matrix are represented visually as Euclidean vectors inside Cartesian coordinate systems. Thus, each vector is displayed as an arrow originating from the coordinate origin ((0,0) in the 2D visualization or (0,0,0) in the 3D visualization), each

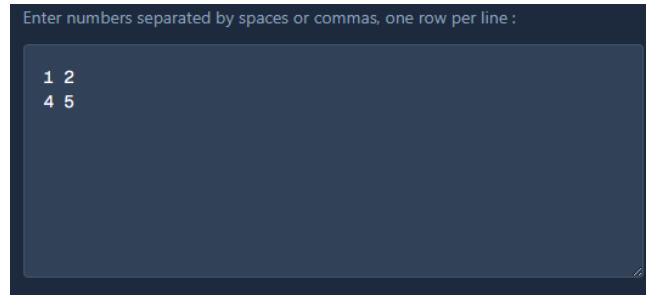


Fig. 5: The input matrix

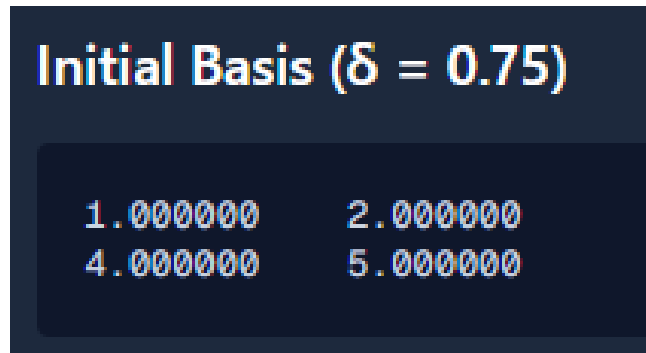


Fig. 6: Basis vectors from the input matrix

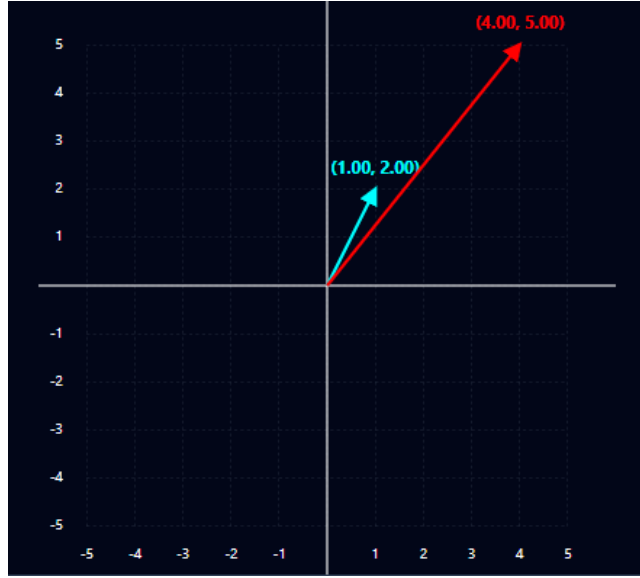


Fig. 7: Basis vectors from the input matrix, represented in the Cartesian coordinate within a Euclidean space

one having a unique color, and each one pointing toward the coordinates which represent their numerical values - or the elements of the matrix. For example, the vector $v_1 = (4, 5)$ is visualized as an arrow from the origin point of $(0, 0)$ toward the point $(4, 5)$. Because all vectors are anchored to the origin point, this allows users to observe the length and direction of these vectors, as well the change of them during the reduction process from the beginning until the final output.

4.2.3 Textual Explanation Panel to Represent the Gram-Schmidt Orthogonalization We decide that the Gram-Schmidt Orthogonalization subprocess would be better off explained through textual explanation in a pop-up window when the user clicks "Show Calculations" button in the "Step and Calculation Details" panel (which we will explain it in the section 4.3.3). At first, the LLL module's frontend source code (`lll/page.tsx`) would call the `runLLL` function from the backend source code of `lll.ts` and inserts the needed parameters - which consist of the basis vectors, the δ -value, and the Boolean flag "true". Within the `runLLL` function, it would call the function `gramSchmidt` and the program would insert the basis vectors to the function and the `gramSchmidt` function would perform a Gram-Schmidt Orthogonalization to these vectors. The function returns the orthogonalized version of these vectors. Later, the function `generateGSOCalculations` would be called and then these orthogonalized vectors, alongside the original vectors, are inserted to the `generateGSOCalculations` function in order to generate a textual display of the mathematical steps behind the Gram-Schmidt Orthogonalization.

4.2.4 Textual Explanation Panel to Represent the Vector Reduction

process Like how the Gram-Schmidt Orthogonalization is shown to the users, we will show the vector reduction process in the form of a textual explanation of its calculation steps in a pop-up window. How we technically display the textual explanation of the calculations behind the vector reduction and vector swap processes are roughly similar to the Gram-Schmidt process explanation. When the frontend called `runLLL` function from the backend source code of `lll.ts`, after performing Gram-Schmidt Orthogonalization and other conditions for the vector reduction to be performed, it would call the function `vectorSubtract` - which as its name suggests, performs a vector size reduction process. The output produced by `vectorSubtract`, together with the previous vector and the currently examined vector would then be inserted into a function named `generateReduceCalculations` in order to generate a textual display of the mathematical steps behind the vector size reduction process.

4.2.5 Textual Explanation Panel to Represent the Vector Swap Pro-

cess If the current basis doesn't satisfy the Lovasz Condition, it wouldn't only triggered a vector reduction, but also the vector swap after the vector reduction. The currently examined vector would have its index getting switched with the index of the previous vector. Later, the function of `generateSwapCalculations` would be executed, which generates a textual display of the mathematical steps behind the vector swap process.

4.2.6 Status Panel to Represent the Lovasz Condition Checking

The Lovasz Condition is depicted as a status panel of two states ("Satisfied" and "Not Satisfied"). Unlike with the usual mechanism of Lovasz Condition checking, for convenience reasons, we keep display the Lovasz Condition status even when the current process isn't the Lovasz Condition checking as in the common LLL algorithm. This is achieved through the addition of the the function `computeLovaszStatus` in the `app/lll/page.tsx` functions to continuously checking the Lovasz Condition status in order to keep the Lovasz Status panel has a status at one time, from the first step of the process until the final step. Obviously we have the regular Lovasz Condition checker - the function `lovaszCondition` in the backend code is to perform Lovasz Condition checking only after the currently examined vector has been reduced. Of course, there is a risk of these two Lovasz Condition checkers would have two different answers at the same time, but we managed to eliminate this risk by employing them into separate responsibilities, as what we already explained before. In the frontend, the `runLLL` would be executed first, then the produced result (`lllResult`) contains data of the steps. The steps would then be used by the `computeLovaszStatus` to visualize the Lovász condition status at each historical step without re-executing the algorithm or affecting the algorithm's control flow. In a shorter explanation, the `computeLovaszStatus` only recheck the Lovasz Status based on the existing data gained from LLL execution by the backend.

4.2.7 Text Panel to Represent Shortest Vector Problem The SVP is computed through each of the BKZ iterations. However, the program would only show it in the final step. It is displayed within a dedicated panel in LLL and BKZ modules.

4.3 Visualization of the LLL Algorithm

First, we have the LLL algorithm. The input panel for this algorithm has matrix and delta value insertion fields, and located in the left half of the page. After the user inserted matrix and delta value properly, and clicked "Process Matrix" button, user can see the "Initial Matrix" as well "Initial Basis" panels below the input panel. To the right half of the page, there is the output section, contains four of five visual and textual information panels, without Shortest Vector Panel.

User can click "Back" and "Next" button in the upper part of the output section to see the previous and next steps of the LLL algorithm process. All visual and textual information panels in the output section are subjects of change according to the current step of the LLL algorithm process.

4.4 Visualization of the BKZ Algorithm

Next, we have the BKZ algorithm, which is one of the more powerful lattice reduction algorithm. Because theoretically this algorithm could be comprised of multiple LLL processes, we design the output section to show the user the number of LLL processes as well an additional information called "Current Process", which provides the user of the steps that is unique to BKZ algorithm, which are the current number of the LLL process and SVP enumeration.

The only differences with LLL page is in the input panel, which has an additional, mandatory field for the block size, and in the output section, in which has Shortest Vector panel. Moreover, in the upper part of the output section, we place four additional information, which are "Block Size", "Delta", "Total LLL Processes", and "Current Process". Out of all of these additional information, only "Current Process" isn't static.

Aside from additional panels and input field, the rest of the visualization and basically mechanism of the output's UI is the same as the LLL ones.

4.5 Output Visualization and Textual Elements for LLL and BKZ

Visualization of both LLL and BKZ algorithms contains following five visual elements in their output section :

4.5.1 Graph Panel

This is the key visual element. People can learn more efficiently if we explain the processes behind both algorithms with a combination of visualization and the textual elements. This is called "Multimedia Effect" (Mayer, RE., 2002). We design the graph panel in 2D and 3D projections, both in the fashion of the



Fig. 8: The 2D visualization



Fig. 9: The 3D visualization

Cartesian coordinate. Each vector arrows represented in this graph panel have different colour, with their vector written in the same colour as their arrow and placed above the tip of their vector arrow.

4.5.2 Lovasz Status Panel

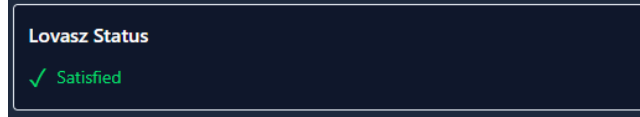


Fig. 10: The Lovasz Condition is displayed in this panel, whether it's satisfied (shown here) or not

Only shown in the LLL module, the panel shows the user of the Lovasz status of the current basis. If the current basis fulfill the Lovasz condition, it will show the user a green check mark and "Satisfied" text, and if not, a red cross mark and "Not Satisfied" text will be shown to the user. Unlike in its original counterpart, the Lovasz Condition is continously checked throughout the whole LLL process, not just after the basis reduction step, for convenience reasons.

4.5.3 Step and Calculation Details

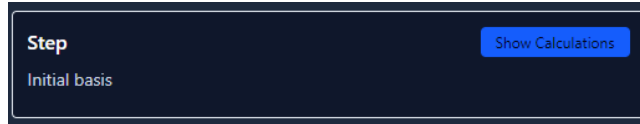


Fig. 11: Current step. Click on the "Show Calculations" for the details

This panel will tell the user of the current step (for example, "Basis size reduction"). The user can see the details of the calculation in the step by clicking "Show Calculations" button at the upper right corner of this panel. There are eight types of LLL and BKZ modules :

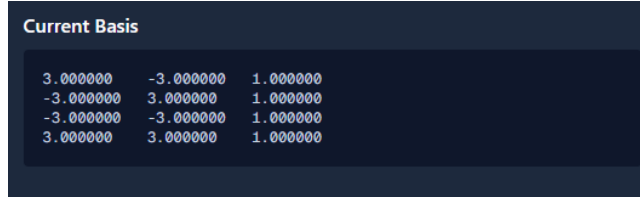
- "Initial Basis". This step is always the first step to be shown after we insert a matrix and clicked "Process Matrix". Right after we clicked the "Process Matrix" button, the program immediately extracted basis vectors from the input. If the "Step" panels shows that the current step is this step, then it means that the program already extracted basis vectors from the input. This step is one of three type of steps who only be shown once, the other two is "Finished" and "Iteration complete" (BKZ-only).

- "Gram-Schmidt orthogonalization". Like its algorithm counterpart, the first "Gram-Schmidt orthogonalization" step would be shown as the second step in a matrix process. In this step, the program performs Gram-Schmidt orthogonalization (GSO) for all vectors from the input, produces the initial orthogonalized $\mathbf{v}_k^*$ and their corresponding projection coefficients $\mu_{k,j}$, with $1 \leq j < k \leq n$. This step also would be executed should a basis doesn't satisfies the Lovasz Condition (in this program, depicted in the form of a panel on its own).
- "Basis reduction". It represents the basis reduction step from the original algorithm, in which a vector that is currently examined (started from the second vector in the basis from the input), having been orthogonalized globally during the Gram-Schmidt orthogonalization step previously together with all other vectors in the basis from the input, is being subtracted by the previous vector, in order to shorten the currently examined vector. This step also would be executed should a basis doesn't satisfies the Lovasz Condition.
- "Basis swap". This step is only executed and shown if during the "real" Lovasz Condition checking (which is executed after the basis reduction step, just like in the original algorithm), the Lovasz Condition isn't satisfied. In this step, like its namesake, the vector which are currently examined, has its index swapped with the previous vector.
- "Start LLL Process $\#n$ ". Only in the BKZ module, when this step is shown, it means that the LLL process in the current block is being executed (in the n -th iteration).
- "Complete LLL Process $\#n$ ". Also only in the BKZ module, when this step is shown, it means that the LLL process in the current block is has been completed (in the n -th iteration).
- "Iteration Complete". Like two previous steps, this is also a BKZ module-only type of step. Like its namesake, if this step is shown, then an iteration is stated to be completed, and unless it's the final iteration, the next iteration begins immediately.
- "Finished". This step is in both LLL and BKZ modules. If this state is shown, then the program (LLL or BKZ, depends on what the module we currently use) terminates. In the context of their respective algorithms, if this step is reached, for LLL, all vectors has been examined and the Lovasz Condition is satisfied for the basis from the input, which in the other words, the basis has been successfully shortened and orthogonalized in accordance to the δ -value. For BKZ, all blocks has been examined (the Lovasz Condition is checked inside each block as part of the local LLL process).

4.5.4 Current Basis Panel Like its namesake, this panel's sole function is to show the user the current basis.

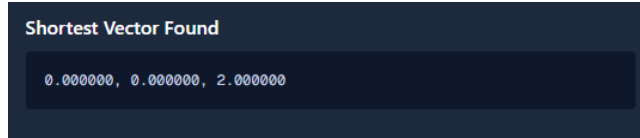
4.5.5 Shortest Vector Panel

This panel's sole function is to inform the user the shortest vectors found in the current basis. Only shown in the last step.



3.000000	-3.000000	1.000000
-3.000000	3.000000	1.000000
-3.000000	-3.000000	1.000000
3.000000	3.000000	1.000000

Fig. 12: The current basis as with the current step



0.000000, 0.000000, 2.000000

Fig. 13: The SVP we gained, which only shown in the final step of the BKZ process

5 Result

5.1 Surveying Method

To measure the quality of Latviz that we have develop, we made an online app usability survey and let the survey participants test the app, by putting the Latviz's access link in the survey form. Since lattice-based cryptography and cryptanalysis algorithms require sufficient understanding of concepts within linear algebra which is usually only taught in colleges, our targeted demographic group of this survey are Indonesian college students and graduates, while the questions in the survey are in the Indonesian language. There are 25 questions related with LLL and BKZ modules in the app (including 2 optional suggestion boxes).

We use Convenience Sampling method as the sampling method for our research, because we already know that both LLL and BKZ, or lattice reduction algorithms in general, require complex mathematical understanding as the requirement for the learners [16]. Linear algebra in general is only start to be taught in colleges, and the fundamental concepts such as linear system of equations, vectors, and matrices are only start to be taught in high schools (in Indonesia, commonly in SMA). Added with time constraints, using convenience sampling method is a primary and a rational choice for us. We spread the survey link to our relatives, especially our fellow faculty colleagues.

First, we asked the participants basic questions about their familiarity with fundamental concepts used in Latviz (for example, post-quantum cryptography) as the background survey of the respondents. Then, for the questions related with the usability evaluations of the LLL and BKZ, we use the 5-points Likert's Scale, which ranges from "Strongly Disagree" as the lowest score to "Strongly Agree" as the highest score. We created the survey using Google Forms, and the

diagrams of the results based on the response from the participants are created automatically by Google Forms.

5.2 Questions

5.2.1 Demographic Background Questions We asked several questions about the respondents' familiarity with cryptography, post-quantum cryptography, lattice-based cryptography, LLL, and BKZ before they trying Latviz for the first time. The answer choices for these questions ranging from "Very familiar" to "Not familiar".

5.2.2 LLL and BKZ Usability Questions For the LLL and BKZ visualization modules in Latviz, we asked the participants about the understanding of LLL and BKZ. For example, the roles of the variables used within each of these algorithms, the changes of the values of these variables, and whether they understand the process of both algorithms from input to the output. We also asked the participants several questions about the effectiveness of the UI and the visualizations of LLL and BKZ to help them understand both algorithms. Lastly, for each LLL and BKZ sections in the survey, we asked the participants to fill the suggestions (which is not mandatory), in which the survey participants can write any suggestions they wish.

5.3 Survey Result

To examine whether these perceptions differed based on prior experience of the survey participants with cryptography, we made a classification of the survey participants based on their self-reported familiarity with cryptography and post-quantum cryptography. We defined the as NC (No Familiarity with Cryptography), LC (Low Familiarity with Cryptography), YC-YPQ (Familiar with Cryptography and Post-Quantum Cryptography), and YC-LPQ (Familiar with Cryptography but Low Post-Quantum Cryptography Knowledge).

Table 1: Respondent Categories

Categories	Percentage
NC	41%
LC	18.1%
YC-YPQ	27.3%
YC-LPQ	13.6%

Table 2: Responses to LLL Evaluation Questions

Statements	SD	D	N	A	SA
After using Latviz, I think I understand LLL better than before	0%	9.1%	18.2%	59.1%	13.6%
The step-by-step visualization with "Back" and "Next" buttons to control the process flow helps me to understand the flow of LLL	0%	4.5%	18.2%	50%	27.3%
It's easy to understand explanations shown in the "Calculation Details"	0%	9.1%	9.1%	45.5%	36.4%
The "Lovasz Status" and "Step" Panels help me enough to understand the ongoing process in LLL	0%	4.5%	18.2%	50%	27.3%
I understand the roles of the variables used in LLL	0%	18.2%	22.7%	36.4%	22.7%
I can track the changes in data or variables at every step of LLL	0%	9.1%	18.2%	45.5%	27.3%
I understand how the input is processed to produce the output	4.5%	4.5%	13.6%	45.5%	31.8%
The visual display is clear and not confusing	0%	9.1%	18.2%	50%	22.7%

Table 3: Responses to BKZ Evaluation Questions

Statements	SD	D	N	A	SA
After using Latviz, I think I understand BKZ better than before	0%	4.5%	22.7%	40.9%	31.8%
The step-by-step visualization with "Back" and "Next" buttons to control the process flow helps me to understand the flow of BKZ	0%	4.5%	13.6%	36.4%	45.5%
It's easy to understand explanations shown in the "Calculation Details"	0%	9.1%	13.6%	45.5%	31.8%
I understand the roles of the variables used in BKZ	0%	13.6%	18.2%	50%	18.2%
I can track the changes in data or variables at every step of BKZ	0%	9.1%	13.6%	45.5%	31.8%
I understand how the input is processed to produce the output	0%	9.1%	9.1%	59.1%	22.7%
The visual display is clear and not confusing	0%	4.5%	13.6%	59.1%	22.7%

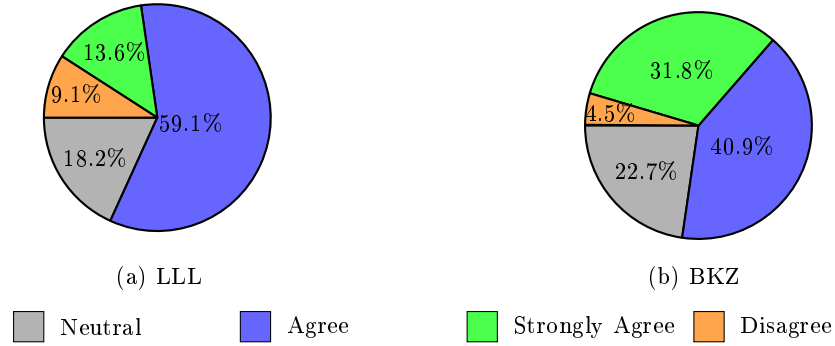


Fig. 14: Responses regarding improved understanding of LLL and BKZ algorithms after using Latviz.

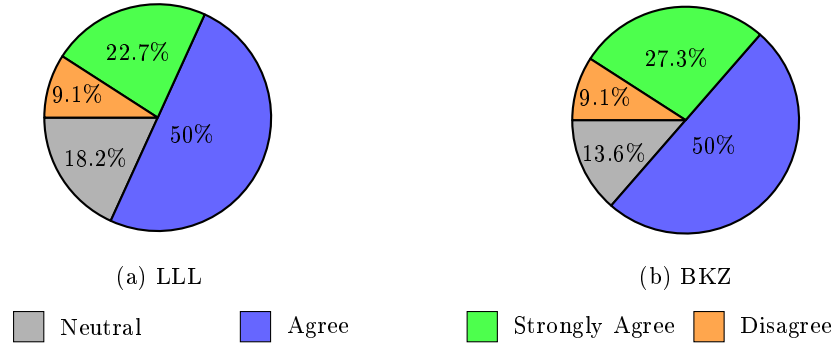


Fig. 15: Responses regarding whether Latviz was perceived as an effective learning tool for LLL and BKZ algorithms

6 Evaluation

Before we closed the survey, we managed to get 22 college students or college graduates as the survey participants. A plurality of all respondents (41% of them) weren't familiar with cryptography, let alone LLL and BKZ. This proves our previous statements about how niche the LLL and BKZ algorithms are.

More than 70% of all respondents reportedly having their understandings of LLL and BKZ algorithms improved after using Latviz's LLL and BKZ visualization modules. This number was taken from the total amount of survey participants who answered "Strongly Agree" and "Agree". Using this response type combination, 72.7% of all survey participants reported understanding increase of LLL and BKZ after they tested Latviz. And of all UI elements surveyed

from both LLL and BKZ visualization modules, for LLL, the most satisfying element is the "Calculation Details" panel (81.9%), and for BKZ, it's none other than the "Back" and "Next" navigation buttons (81.9%).

The less positive responses are found in questions about the roles of the variables used in both algorithms. In particular, just 59.1% of all respondents understand the roles of the variables used in the LLL algorithm, and for BKZ, it is slightly lower, at just 58.2%. We think this is natural consider how hard the concepts of LLL (and BKZ) are.

Curiously, despite being mathematically more complex compared to LLL, the Latviz's BKZ visualization module, according to this survey, is slightly better in terms of visualization and navigation control. LLL is better compared to BKZ only in terms of understanding increase after testing the Latviz app. Which means that BKZ's visualization is better than LLL's visualization, yet LLL is still easier to be understand compared to BKZ, which is quite a very expected result.

7 Conclusion

From the results of our survey, overall, LLL and BKZ visualizations provided by Latviz help most users to increase the understandings of the LLL and BKZ algorithms. The BKZ visualization is better compared to LLL's, yet BKZ is still harder to understand compared to LLL. The keys of these positive remarks about BKZ (and LLL) are their UIs.

We conclude that according to our own survey, while most of the response were more positive than negative, it is still a necessary to improve our Latviz, especially for aspects that are supposedly intended to help users easier to understand the algorithms we want to teach. For example, language options and the calculation details. Overall, the results are mostly positive with a need for more improvement in the future to help the users understand these algorithms easier.

8 Future Works

Based on the response from the survey participants, we would like to add other languages, like Indonesian. We would also prioritize to improve the calculation explanation to make them more understandable for users. For the input aspect, we would like to add another input mechanism that can allow users to upload existing matrices from CSV or PNG files. The input form choice (which, as of now only matrices) itself is also can be diversified by developing a mechanism to extract vectors from inputs within the form of systems of linear equations. And lastly, we would like to add optional colour schemes of Latviz's UI.

References

1. Aiman Al-Sabaawi. (2021) Cryptanalysis of Classic Ciphers: Methods Implementation Survey. In Proceedings of the 2021 International Conference on Intelligent

- Technologies. Institute of Electrical and Electronics Engineers Inc., United States of America, pp. 1-6.
2. Meyer Alexander. (2024). Post-Quantum Cryptography: An Analysis of Code-Based and Lattice-Based Cryptosystem. arXiv:2505.08791.
3. Nguyen, Phong. Q, Vallée, B. (2010). The LLL Algorithm: Survey and Applications.
4. Schnorr, C. P., Euchner M. (1994). Lattice Basis Reduction: Improved Practical Algorithms and Solving Subset Sum Problems.
5. FPLLL Development Team. (2023). fplll, A Lattice Reduction Library Version 5.5.0. Github. <https://github.com/fplll/fplll>
6. Bennett, Zachary R. (2026). lll-attack-runner. Github. <https://github.com/curiosityz/lll-attack-runner>
7. SageMathCell Development Team. (2011). SageMathCell Version 1. <https://sagecell.sagemath.org>
8. Mayer, Richard E., (2001). Multimedia Learning. Cambridge University Press.
9. Hundhausen D. Christopher, Sarah A. Douglas, John Stasko. (2002). A Meta-Study of Algorithm Visualization Effectiveness. Journal of Visual Languages and Computing, 13(3), 259-290.
10. Konyalioglu A. Cihan, Ahmet Isik, Abdullah Kaplan, Seyfullah Hizarci, Merve Durkaya. (2011). Visualization approach in teaching process of linear algebra. Procedia Social and Behavioral Sciences 15 (2011) 4040-4044.
11. Yunfei Wang, Jin Xin, Liu Junyu. (2025). Learning with errors may remain hard against quantum holographic attacks arXiv:2509.22833
12. Silverman, Joseph H. (2020). An Introduction to Lattices, Lattice Reduction, and Lattice-Based Cryptography. IAS/Park City Mathematics Series Graduate Summer School Lecture Notes.
13. Zhao Ziyu, Ding Jintai. (2022). Practical Improvements on BKZ Algorithm. National Institute of Standards and Technology <https://csrc.nist.gov/csrc/media/Events/2022/fourth-pqc-standardization-conference/documents/papers/practical-improvements-on-bkz-algorithm-pqc2022.pdf>
14. Jianwei Li, Phong Q Nguyen. (2024). A Complete Analysis of the BKZ Lattice Reduction Algorithm. Journal of Cryptology, In press, [10.1007/s00145-024-09527-0](https://doi.org/10.1007/s00145-024-09527-0). [hal-04728596](https://doi.org/10.1007/s00145-024-09527-0)
15. Calderone, Dan. (2023). Vector Visualizations. Github. <https://danjcalderone.github.io/papers/vectors.pdf>
16. Surin, J., Cohney, S. (2023). A Gentle Tutorial for Lattice-Based Cryptanalysis (Report No. 2023/032). Cryptology ePrint Archive. <https://eprint.iacr.org/2023/032>
17. Jaziel D. Flores Rodríguez, Miguel E. Pérez Ibarra, Fernando Q. Valencia Rodríguez, Alfonso F. De Abiega L'Eglise, Gina Gallegos García (2024). Herramienta de software para la visualización de la criptografía basada en retículas. POLIBITS, 66(1), 41-48.